B.E. PROJECT · STAGE II · IMPLEMENTATION PLAN

Privacy-Preserving Money Laundering Detection Across Heterogeneous Banks Using Federated Learning and Blockchain

Revised scope, architecture, data plan, phase-wise timeline, experiments, team split and concepts to master

Team	Anuj Nanhe (23) · Aniket Shelke (58) · Laiba Shaikh (62)
Guide	Dr. S. R. Khonde
Institute	MES Wadia College of Engineering, Pune · Dept. of Computer Engineering
Academic year	2026–27
Document date	17 October 2026 · working draft for team discussion

NOTE ON THE TITLE

The title above is a *suggested* revision reflecting the new banking-only scope. The Stage-I title referred to Trade-Based Money Laundering (TBML), which is now out of scope. Any change of title or scope must be confirmed with the guide.

01 / REVISED SCOPE

What changed since Stage I

- **Trade-based aspect dropped.** The project now focuses on banking transaction-level money-laundering detection (AML). Invoice, shipment and customs data are out of scope.
- **FL design corrected.** Stage I paired FedAvg with clients holding different kinds of data (bank payments, trade records, customs). FedAvg assumes clients share the same model shape, so we now use **heterogeneous horizontal FL**: private bank-specific encoders plus shared layers aggregated with FedAvg.
- **Dataset chosen.** IBM Transactions for Anti-Money Laundering (HI-Small), plus a small synthetic layer of bank-specific customer profiles to create genuinely different schemas.
- **Blockchain kept, but focused.** It is an audit and accountability layer, not part of detection itself.

REVISED RESEARCH QUESTION

Can banks with heterogeneous data schemas collaboratively train a money-laundering detection model via Federated Learning, without sharing raw data, while maintaining a tamper-evident audit trail of training and alerts?

Guiding principle

Every component must be finishable by a three-person team, demoable end to end, and defensible in the viva. Where a more advanced option exists, it is listed as a stretch goal rather than placed on the critical path.

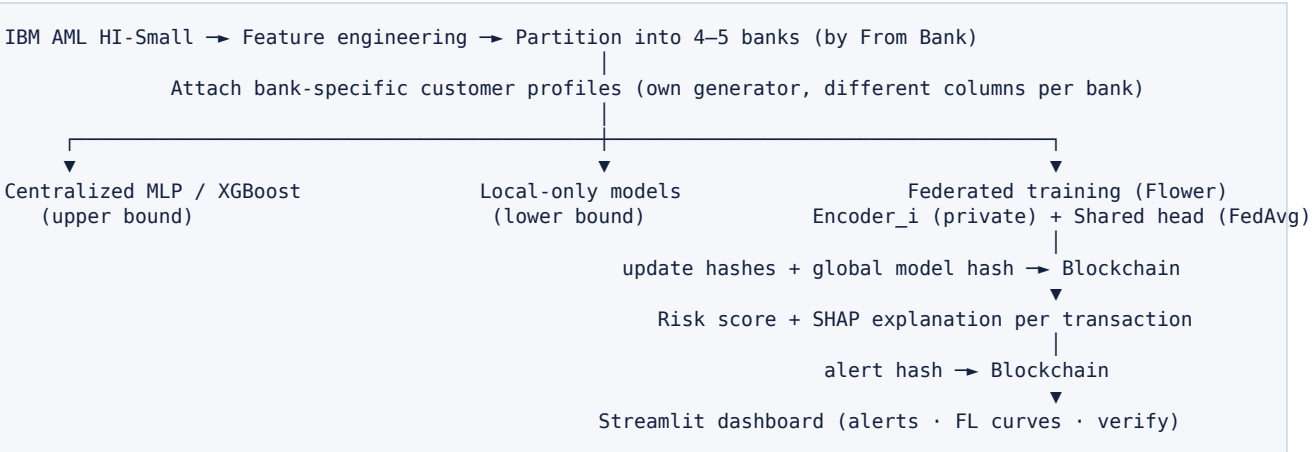
02 / DESIGN DECISIONS

Final technology and design choices

Component	Choice	Reason
Domain	Banking AML, transaction-level detection	Clean scope; public labelled data exists
Base dataset	IBM AML dataset, HI-Small variant (Kaggle)	Peer-reviewed synthetic data (Altman et al., NeurIPS 2023); *From Bank* / *To Bank* columns allow partitioning by real bank IDs
Schema heterogeneity	Own generator for bank-specific customer profiles	Each bank gets columns the others don't have; weak label correlation avoids circularity
FL type	Heterogeneous horizontal FL: private encoders + shared head, FedAvg on shared layers	Handles different schemas and uses FedAvg correctly (LG-FedAvg / FedPer family)
FL framework	Flower (flwr), simulation mode	Mature, Python, PyTorch-friendly; same code can run in deployment mode later
Model	PyTorch MLP on engineered tabular features	Fast, simple, and averageable (tree models are not)
Blockchain	Hardhat (or Anvil) + Solidity + web3.py; access-controlled contract	Quick to set up; Fabric is heavier than the payoff. Ganache/Truffle are discontinued
Explainability	SHAP on each bank's full local model	Explanations come out in each bank's own schema
Dashboard	Streamlit	Pure Python, quick to build
Stretch only	Vertical FL, Differential Privacy, Secure Aggregation	Valuable, but not needed to answer the research question

03 / ARCHITECTURE

End-to-end pipeline

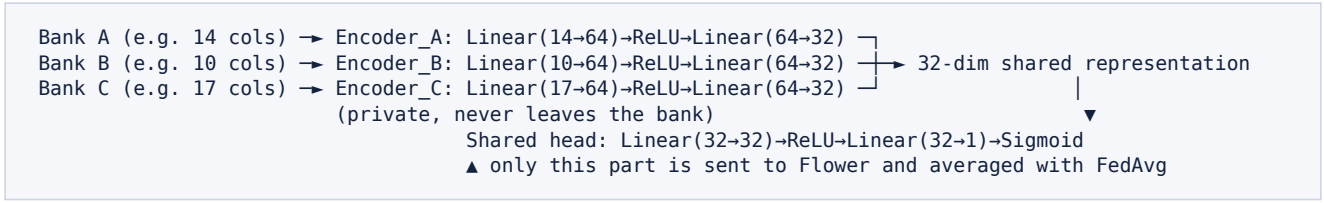


KEY ARCHITECTURAL PRINCIPLE

Raw data and bank-specific encoders never leave the bank. Only the shared layers' parameters travel to the server. Only hashes and metadata go on-chain.

04 / MODEL

Model design: private encoders + shared head



- Each bank's encoder maps its own schema into a common 32-dimensional representation. Its input size can differ per bank.
- The shared head has identical shape everywhere, so FedAvg is mathematically valid on it.
- Related methods to cite: **LG-FedAvg** (Liang et al., 2020, “Think Locally, Act Globally”), **FedPer**, **FedRep** (partial or personalized parameter aggregation).
- In Flower, the client's `get_parameters()` / `set_parameters()` return and load only the shared-head weights.

VIVA ONE-LINER

“Banks have different schemas, so each bank keeps a private encoder that maps its data into a shared representation, and only the shared layers are aggregated with FedAvg using Flower.”

05 / DATA

Data plan

5.1 Base transactions: IBM AML HI-Small

- Synthetic but realistic, labelled (*Is Laundering*), widely cited, so results can be compared with published work.
- Columns include timestamp, From Bank, sender account, To Bank, receiver account, amounts paid and received, currencies, and payment format.
- Laundering patterns in the data include fan-in, fan-out, cycles, scatter-gather, gather-scatter, bipartite and stack structures.
- **Partitioning:** group transactions by *From Bank* into 4–5 client banks. This gives naturally non-IID clients.
- **Split by time** (train / validation / test), not randomly, to avoid leakage from future transactions.
- If the full file is slow during development, subsample (e.g. about 500k transactions) and use the full set for final runs.

5.2 Our layer: bank-specific customer profiles

We generate synthetic customer attributes per account, with a **different set of columns at each bank**. This turns schema heterogeneity from a simulation of column subsets into genuinely different schemas.

Bank	Example bank-specific columns
Bank A	account_age_days, customer_type (individual / business), kyc_risk_level
Bank B	occupation_category, declared_annual_income, branch_region
Bank C	account_type, num_linked_accounts, onboarding_channel
Bank D / E	Mix or subset of the above, with different categorical encodings

- Keep the correlation between these attributes and the laundering label **weak** (e.g. laundering accounts slightly more likely to be new or to have income mismatches). The main signal must still come from IBM's transaction patterns.
- **Why:** this avoids circularity, i.e. a model that only detects patterns we injected ourselves.
- **Ablation:** run experiments with and without the synthetic attributes and report both.

HOW TO DESCRIBE IT IN THE REPORT

"Transactions are from the IBM AML dataset (HI-Small). Bank-specific customer attributes were synthetically generated to simulate schema heterogeneity; their correlation with labels was deliberately kept weak."

5.3 Engineered transaction features (shared pool)

Group	Features
Transaction	log amount, payment format, currency, cross-currency flag, cross-bank flag, round-amount flag, hour, day of week
Sender account	transaction count, mean / std of amount, fan-out (unique receivers), number of currencies used, time since previous transaction
Receiver account	fan-in (unique senders), total received volume, burstiness (transactions in last 24 h)

All of these can be computed with pandas `groupby`, so no heavy graph library is required. Fan-in and fan-out capture the mule and scatter-gather behaviour that matters most in this dataset. Compute account-level aggregates using **past transactions only** to avoid leakage.

5.4 Class imbalance

- Laundering is a tiny fraction of transactions.
- For training only: undersample normal transactions (around 1:20) and use a class-weighted BCE loss.
- Always evaluate on the untouched, imbalanced test split. Tune the decision threshold on validation data.
- Main metric: **PR-AUC**, never plain accuracy.

06 / BLOCKCHAIN

Blockchain audit layer

Why a blockchain and not a signed database log? Several mutually distrusting banks collaborate, and no single party, not even the FL server, should own or be able to rewrite the audit log. A shared ledger that no single bank controls fits that situation.

Three jobs of one smart contract

Function	What is recorded	Who calls it
Bank registry	Registered bank addresses; only these can write (makes the chain permissioned)	Admin, once at setup
FL round audit	Each bank's update hash per round; server's global-model hash per round. Clients verify the received model against the chain	Flower client fit(); server strategy aggregate_fit()
Alert log	hash(txn_id + risk score + top SHAP features + model version) + timestamp	Scoring script / dashboard

Hook points in Flower

```
Bank client (fit)                          Server (custom Strategy)
─────────────────                          ────────────────────────
train shared layers locally
hash(update) ──► contract.submitUpdate()
send update to server ──────────────────►  aggregate_fit(): FedAvg
                                           hash(global) ──► contract.registerGlobalModel(round, hash)
receive global model ◄──────────────────   send global model
verify hash == on-chain hash
```

Minimal contract sketch

```
contract FLAudit {
    mapping(address => bool) public registeredBanks;
    struct Round { bytes32 globalHash; uint256 timestamp; }
    mapping(uint256 => Round) public rounds;
    mapping(uint256 => mapping(address => bytes32)) public updates;
    mapping(bytes32 => uint256) public alerts;   // alertHash => timestamp

    modifier onlyBank() { require(registeredBanks[msg.sender], "not registered"); _; }

    function submitUpdate(uint256 r, bytes32 h) external onlyBank { updates[r][msg.sender] = h; }
    function registerGlobalModel(uint256 r, bytes32 h) external { rounds[r] = Round(h, block.timestamp); }
    function logAlert(bytes32 h) external onlyBank { alerts[h] = block.timestamp; }
}
```

Still to add: registerBank(), events for every write, and access control so that only the server address can call registerGlobalModel().

Demos and measurements

- **Tamper demo 1:** edit a stored alert, then show that verification fails.
- **Tamper demo 2:** the server sends a modified global model; the client rejects it because the hash doesn't match.
- **Measure:** latency per on-chain write, extra time per FL round (with vs without chain), gas per operation, storage growth.

LIMITATION TO STATE HONESTLY

The blockchain proves records were not changed *after* logging. It does not prove they were correct *when* logged: a malicious bank can still register the hash of a poisoned update.

07 / TIMELINE

Implementation phases

Assumption: groundwork from now through December (around exams), core build January–March, evaluation and report March–April. Adjust to the college calendar.

Phase	Duration	Work	Milestone
0. Setup & learning	2 weeks (Oct–Nov)	GitHub repo, shared environment; everyone completes the Flower PyTorch quickstart and a Hardhat hello-contract; download and explore HI-Small	Flower quickstart runs; a contract is deployed locally and called from Python
1. Data & features	2–3 weeks	Cleaning, feature pool, time-based split, bank partitioning, profile generator for bank-specific columns	One Parquet file per bank + held-out test set
2. Baselines	2 weeks	Rule-based baseline; centralized Logistic Regression, XGBoost, MLP; local-only MLP per bank	Baseline results table (PR-AUC, F1, recall, FPR)
3. Federated learning	3–4 weeks	Step 1: naive FedAvg on common features. Step 2: encoder + shared-head model. Experiments on rounds, clients, local epochs, FedAvg vs FedProx; log bytes per round	FL results table + training curves
4. Blockchain	2–3 weeks (parallel with 3)	Contract + Hardhat tests; web3.py wrapper; hooks in Flower client and server; tamper demos	FL run with on-chain round logs + latency numbers
5. Explainability & dashboard	2 weeks	SHAP on flagged transactions per bank; Streamlit pages: alerts, FL curves, blockchain verify	Working end-to-end demo
6. Evaluation & report	3–4 weeks	Final experiments, results tables, threat model, limitations, report, paper draft if required	Final report and presentation

08 / EVALUATION

Experiments and metrics

#	Experiment	Question it answers
1	Centralized vs local-only vs naive FedAvg vs encoder-FedAvg	Does collaboration help, and how close does FL get to pooled data?
2	Common-features FedAvg vs encoder-FedAvg	Does handling schema heterogeneity help?
3	With vs without synthetic profile attributes	Do results depend on the attributes we generated? (circularity check)
4	Number of clients (3, 4, 5) and local epochs	How sensitive is FL to its settings?
5	FedAvg vs FedProx	Effect of non-IID data across banks
6	Bytes transferred per round	Communication overhead of FL
7	Round time with vs without blockchain; gas per tx; storage growth	Blockchain overhead
8	Tamper-detection demos	Does the audit layer work?

- **Detection:** Precision, Recall, F1, **PR-AUC** (main), false-positive rate, confusion matrix, recall per laundering pattern.
- **System:** bytes per round, wall-clock time per round, latency (ms) per on-chain write, gas per operation.
- Fix random seeds and report mean ± std over at least 3 runs for the main comparisons.

09 / TEAM

Team split

Member	Owns	Supports
Anuj Nanhe	Data pipeline, feature engineering, profile generator, baselines, SHAP	Evaluation
Aniket Shelke	Flower FL (naive and encoder FedAvg), FL experiments	Model design
Laiba Shaikh	Smart contract, web3.py integration, Streamlit dashboard	Tamper demos

Everyone writes the report sections for their own module and must be able to explain the whole pipeline in the viva.

10 / CODEBASE

Repository structure

```
aml-fl/
├── data/            raw/, processed/, partitions/
├── notebooks/       eda.ipynb, baselines.ipynb
├── src/
│   ├── features.py        feature engineering
│   ├── profiles.py        bank-specific customer profile generator
│   ├── partition.py       bank split + per-bank schemas
│   ├── models.py          Encoder, SharedHead, BankModel
│   ├── baselines.py
│   ├── fl/                client_app.py, server_app.py, strategy.py
│   ├── chain/             contracts/FLAudit.sol, chain_client.py (web3)
│   └── explain.py         SHAP
├── dashboard/       app.py (Streamlit)
├── experiments/     configs + results CSVs
└── report/
```

11 / REQUIREMENTS

Requirements

Type	Requirement
Software	Python 3.10+, pandas, NumPy, scikit-learn, XGBoost, PyTorch, Flower (flwr), SHAP, Streamlit, web3.py, Node.js + Hardhat, Solidity, Git / GitHub, Jupyter
Hardware	Laptop with 8–16 GB RAM; no GPU needed (small tabular models). Google Colab as backup
Functional	Load and partition data; generate bank profiles; train baselines and federated models; score transactions; explain alerts; register and verify hashes on-chain; show results on a dashboard
Non-functional	Raw data never leaves a bank; bank encoders never leave a bank; tamper-evident audit trail; reproducible experiments (fixed seeds, saved configs); acceptable latency per verification

12 / KNOWLEDGE

Concepts to know

Before starting (everyone)

- **ML basics:** train / validation / test, overfitting, Logistic Regression, decision trees, Random Forest, gradient boosting, neural-network training loop (loss, gradient descent, epochs, batches).
- **Imbalanced classification:** why accuracy misleads; precision, recall, F1, PR-AUC, confusion matrix, false-positive rate.
- **Hashing:** what SHA-256 is and why any change to the input changes the hash.
- **Blockchain basics:** blocks, immutability, consensus, public vs permissioned chains, smart contracts.
- **AML domain:** placement, layering, integration; money mules; fan-in / fan-out; structuring (smurfing).

To explain the implementation

Who	Must be able to explain
Everyone	FedAvg (weighted-average formula); horizontal vs vertical FL and why ours is heterogeneous horizontal FL; why PR-AUC over accuracy; what is and is not on-chain; the end-to-end pipeline
FL owner	Partial parameter aggregation (LG-FedAvg / FedPer idea); non-IID data and FedProx; Flower's ClientApp / ServerApp / Strategy model; gradient leakage and why Secure Aggregation and DP exist
Data owner	Laundering patterns in the IBM data; feature-engineering logic and leakage avoidance; profile generator and the circularity argument; imbalance handling; SHAP / Shapley values
Blockchain owner	Hashing, smart contracts, gas, events, access control; permissioned vs public chains; why a blockchain over a signed log; its limitation (integrity after logging, not correctness)

13 / PREPARATION

Likely viva questions

- Why Federated Learning instead of encrypting and pooling the data?
- Your schemas differ. How can FedAvg work? (Private encoders, shared head.)
- Isn't the IBM data synthetic? How do you know the results transfer to real banks?
- You generated the customer profiles yourselves. Doesn't that bias the results? (Weak correlation + ablation.)
- What happens if one bank sends a malicious update?
- Why a blockchain instead of a signed database log?
- How much performance do you lose with FL compared with centralized training, and why?

14 / BEYOND THE CORE

Stretch goals and risks

Stretch goals (only after the core demo works)

- Secure Aggregation using Flower's SecAgg+.
- Differential privacy (e.g. Opacus) with a privacy vs accuracy curve.
- Vertical FL demo: sender-bank / receiver-bank split on the same transactions, adapted from Flower's official vertical-fl example (split learning, not FedAvg).

- Poisoning-attack experiment showing how the round audit trail localizes the bad round.

Risks and mitigations

Risk	Mitigation
Dataset too large or slow	Subsample (about 500k transactions) for development; full set for final runs
FL underperforms centralized	Tune local epochs and learning rate; try FedProx; report honestly (a gap is an expected finding)
Blockchain integration eats time	One contract, three functions; build in parallel from the start
Very low recall due to imbalance	Weighted loss, threshold tuning on validation, PR-AUC as main metric
Feature leakage from future data	Time-based split; account aggregates computed from past transactions only

15 / KEY REFERENCES TO ADD

New references for the revised scope

- E. Altman et al., “Realistic Synthetic Financial Transactions for Anti-Money Laundering Models,” NeurIPS 2023 (Datasets & Benchmarks). Source of the IBM AML dataset.
- P. P. Liang et al., “Think Locally, Act Globally: Federated Learning with Local and Global Representations,” 2020 (LG-FedAvg).
- M. G. Arivazhagan et al., “Federated Learning with Personalization Layers,” 2019 (FedPer).
- H. B. McMahan et al., “Communication-Efficient Learning of Deep Networks from Decentralized Data,” AISTATS 2017 (FedAvg).
- T. Li et al., “Federated Optimization in Heterogeneous Networks,” MLSys 2020 (FedProx).
- D. J. Beutel et al., “Flower: A Friendly Federated Learning Research Framework,” 2020.
- Keep Stage-I references [3]–[16]; drop or de-emphasize the TBML-specific papers [1] and [2].